Project Proposal: 2D CAD-Integrated CFD Solver

Nivaan Kaushal
October 19, 2026

1. Project Overview

The primary objective of this project is to build a 2D Computational Fluid Dynamics (CFD) solver in Python without using major libraries that perform calculations natively. The tool will parse 2D cross-sectional CAD geometries (STEP or coordinate profiles) to evaluate flow fields, surface pressure distributions, and boundary layer behavior. If possible, extensions to 3D geometry may be explored.

2. Learning Goals

By the end of this project, I aim to build strong familiarity and proficiency in:

- **Navier-Stokes Equations:** Physical implications, mathematical formulations, and engineering applications.
- **Numerical Methods:** Spatial/temporal discretization, boundary condition implementation, and solver stability in CFD.
- **Code Optimization:** Efficient matrix operations, sparse linear solvers, and vectorization for reduced compute time.
- **Core Fluid Dynamics:** Overall understanding of relevant quantities and concepts.

3. Technical Approach & Reference

The computational framework will build upon Prof. Lorena Barba's 12 Steps to Navier-Stokes:

- **Theoretical Foundation:** Master finite difference schemes, CFL stability criteria, and Poisson solvers.
- **Geometry Setup:** Parse 2D boundary nodes using `trimesh/shapely` to generate structured profile grids.
- **Governing Equations:** Implement 2D incompressible Navier-Stokes in streamfunction-vorticity ($\psi - \omega$) form: $\frac{\partial \omega}{\partial t} + u\frac{\partial \omega}{\partial x} + v\frac{\partial \omega}{\partial y} = \nu\left(\frac{\partial^2 \omega}{\partial x^2} + \frac{\partial^2 \omega}{\partial y^2}\right)$, $\nabla^2 \psi = -\omega$
- **Boundary Layer Coupling:** Combine inviscid outer flow solutions with integral boundary layer methods (δ^*, C_f).

4. Milestone Roadmap

Phase	Core Deliverable & Scope	Timeline
1	**Study & Initial Prototyping:** Deep dive into Navier-Stokes and finite difference methods using `CFDPython` (Steps 1–10).	Weeks 1–2
2	**Core Solver Development:** Build 2D Navier-Stokes solver and integrate 2D CAD boundary extraction.	Weeks 3–5
3	**Boundary Layer Module:** Implement integral boundary layer calculations (skin friction C_f and separation dynamics).	Weeks 6–8
4	**Validation & Benchmarking:** Validate results against standard flat plate and airfoil experimental/numerical datasets. Make adjustments to model accordingly.	Weeks 9–10
5	**Final Adjustments:** Make final adjustments and modifications. Potentially develop UI for enhanced user experience and simulation visualizations.	Weeks 11–12

5. Computational Framework

An emphasis will be placed on minimal non-native library usage to maintain full visibility into the underlying numerical routines. Core scientific libraries will be leveraged as follows:

- **Numerical Computations & Matrix Operations:** `NumPy` for vectorized array manipulations and finite difference stencils.
- **Sparse Linear Solvers & Differential Equations:** `SciPy` (`scipy.sparse` and `scipy.linalg`) for fast Poisson equation solves and system integration.
- **Geometry & Mesh Processing:** `trimesh` and `shapely` for parsing 2D STEP/STL cross-sections, extracting node points, and evaluating surface normals.
- **Visualization & Analysis:** `Matplotlib` for inline vector plots and surface pressure profiles; `PyVista` or `VTK` exports if 3D geometry visualization is explored.